



CAB420 ASSESSMENT 1A REPORT

RORY BLAIR, N10773398



APRIL 6, 2025

CAB420
MACHINE LEARNING

PROBLEM 1:

Pre-Processing:

Processing data is an important part when preparing a model for training, especially models used for predicting an outcome. The data was collected from their respective '.csv' files, separated into their respective sets and then standardized. The standardization of the data sets was completed using a function set out within the week 2 practical [1]. Although standardization is not required for linear regression, it is necessary for the ridge and lasso models as both models are sensitive when it comes to the amount of the data. Standardizing the data prevents bias towards certain variables. [2] No other processing such as removal of outliers was not undertaken as so could result in bias. Therefore, the standardised data is used for all 3 models as to make sure models are tested on the same data.

Selection of Lambda values:

The selection of the lambda values is very important in the ridge and lasso models as the lambda value is used to ensure a balance between overfitting and underfitting by managing variance and bias in the model's coefficient. A range of value from 10^{-10} to 1 were selected as appropriate values for lambda, which is a common range to begin with and considered a 'coarse search'. This is the same range of lambda values given except the number of values created has been increased from the default, 50, to 100 to allow more values to be tested. This range of lambda values allow for experimentation of strong and weak regularisation, thus allowing balance between overfitting and underfitting. If done correctly, a value of lambda would be found that does not lose its prediction accuracy by lowering the Root Mean Square Error (RMSE) on the validation set.

Compare 3 Models:

All 3 models were trained and evaluated using the same standardized data. The linear regression model was borrowed from the week 1 practical [3] whilst the ridge and lasso models were borrowed from the week 2 practical, specifically Questions 2 and 3. [1] The evaluation methods were also borrowed from these weeks allowing the exploration of a range of different lambda values and finding the best lambda values and there corresponding measures of fit. A comparison of the 3 models can be seen below in Table 1.1. This tables shows the lambda values, RMSE, R^2 and Adjusted R^2 (See Appendix 1D, 1E & 1F for model summary table)

	Linear Regression	Ridge	Lasso
Lambda	NONE	0.061	0.002
RMSE	0.730	0.142	0.144
R²	0.759	0.718	0.706
Adjusted R²	0.637	0.575	0.557

Table 1.1: Problem 1 Model Comparison

As seen in table 1.1 above, the linear regression model has the highest values in all 3 measures of fit, whilst the ridge and lasso models appear to be very similar. As note the R^2 and R^2 Adjusted values are high indicating that the model is good at describing the data variance. The only problem with the linear regression model is that it has a very high RMSE value suggesting the model would making mistakes, thus overfitting to the training data and affecting its prediction accuracy [4].

Although the ridge and lasso models don't have higher R^2 and R^2 Adjusted values they are still reasonable values. Unlike the linear regression model and it's high RMSE values, both the ridge and lasso models have considerable low RMSE values. Low RMSE values mean that the model makes more accurate predictions [4]. The differences in prediction accuracy can be visually represented as residual plots as seen in Figure 1.1, 1.2 and 1.3 below.(See Appendix 1A for visual similarities in R^2 values via Q-Q plots) Between Ridge and Lasso the models seemed to be partially accurate when it comes to the Test Set performance (See Appendix 1B for test performance for ridge and Appendix 1C for test performance for lasso)

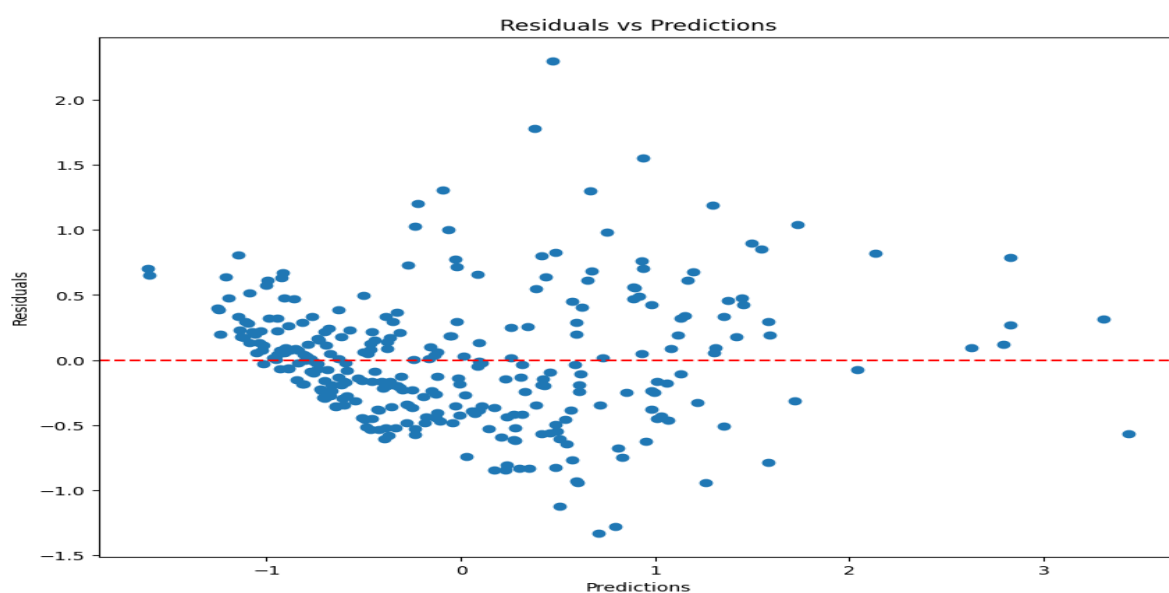


Figure 1.1: Plot of Residuals for Linear model

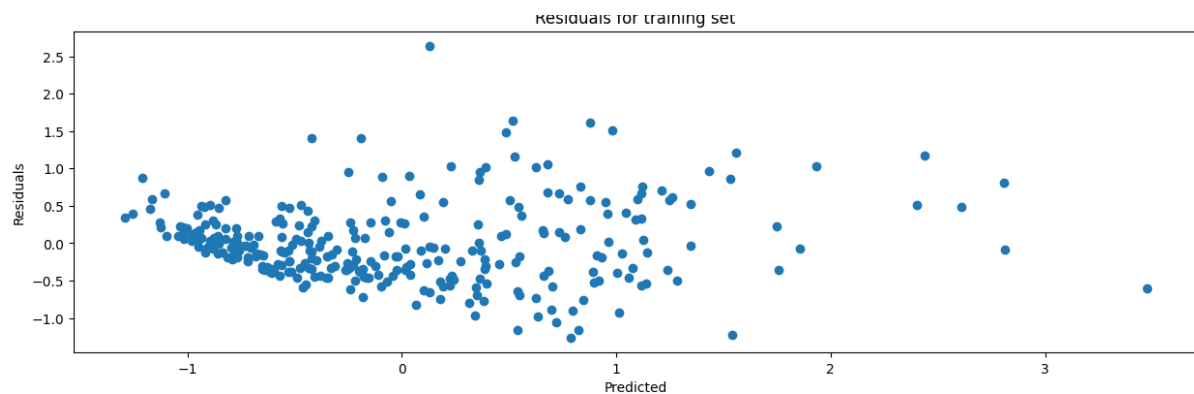


Figure 1.2: Plot of Residuals for Ridge model

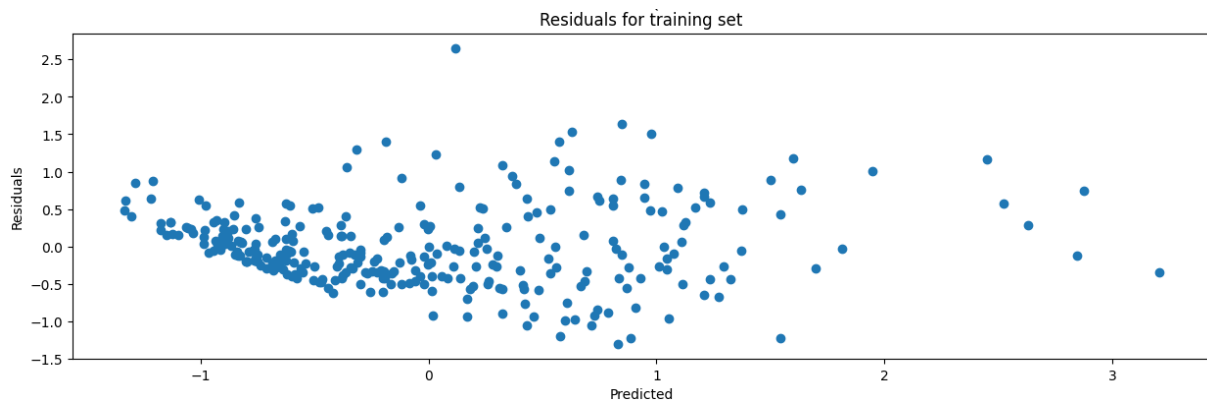


Figure 1.3: Plot of Residuals for Lasso model

One testing assumption that can be made is about the residual distribution, which was tested using both Q-Q plots and more precisely residuals for training sets. As seen in Figure 1.1, 1.2 and 1.3 the plots show no clear patterns meaning that the residuals were reasonably distributed. Although the residuals were reasonably distributed the RMSE value for the linear model was high, suggesting that it didn't work well.

Ethical Concerns:

When creating prediction models, topics such as crime, demographics, and socio-economic need ethical considerations when determining the results of a prediction model. One of the most important considerations is the fairness/bias of the results. For example, the data could have biased data towards certain communities, thus the models could make biased/unfair prediction. The final consideration is the model accountability and making sure the model is validated and monitored so that biased data cannot cause serious consequences, such as misallocation of resources and false accusations. [5]

PROBLEM 2:

Pre-Processing:

Unlike the 1st problem, no additional data processing was required as the data provided was already extracted and split into their respective data set. These data sets do not include missing values, are all numerical and is already useable. No scaling or normalisation was performed as to improve performance. Scaling is not required as the range of data is satisfactory. Normalisation was not necessary as undesirable data, such as outliers, should be included as to provide information about how well the model distinguishes the classes. Removing the undesirable data could lead to crucial information being falsified as the model will not include the few outliers to really test the models.

Hyper-parameter selection:

All 3 models used a 'gridSearch' method and 'trial & error' method to find hyper-parameters. The 'gridSearch' method is used to find optimal hyper-parameters based on the parameter that are tested within the 'gridSearch'. This method was borrowed from the week 3 practical [6]. The 'trial & error' method worked by starting off with testing one hyper-parameter, finding the best option and then testing with that optimal hyper-parameter and a different hyper-parameter. This method was repeated until an optimal value was found for all hyper-parameters.

For the K-Nearest Neighbour (KNN), there were 3 hyper parameters that were tested, which are shown in table 2.1 below. Usually if the 'n_neighbours' value is small it can lead to over fitting, whilst a larger value could lead to underfitting [7]. This is the reason the list of neighbours starts at 3 as if it started at 1 or 2, they were chosen, thus overfitting the model.

Hyper-Parameter	Options
n_neighbours	list(range(3,198))
Weights	'uniform' 'distance'
Metric	'minkowski' 'manhattan'

Table 2.1: KNN Hyper-Parameter Option

For the Support Vector Machine (SVM) there were 4 hyper parameters that were tested, which are shown in table 2.2 below. Usually a smaller ‘C value’ put more attention on maximising the margin instead of classifying the training correctly, meaning there is more errors in classification. On the other hand if ‘C value’ is large the opposite happens. However, this can lead to overfitting, which is the reason the tested ‘C value’ range from 0.01 to 1000.

Hyper-Parameter	Options
C	[0.01, 0.1, 1, 10, 100,1000]
Kernel	'linear' 'rbf' 'poly'
Gamma (IF Kernel = rbf)	[1, 0.1, 0.01, 0.001]
Degree (IF Kernel = Poly)	[1,3, 4, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37, 39]

Table 2.2: SVM Hyper-Parameter Option

For the Random Forest (RF) there were only 2 hyper parameters that were tested, which are shown in table 2.3 below. The ‘n_estimator’ the number of trees in the forest, meaning the larger the number of trees, the more accurate the model. But the larger number usually means more computational power, thus a longer training process. The depth value is the depth that each tree will explore and works similar to the ‘n_estimator’. The larger the depth, the more accurate the model, but a longer and slower training process [8].

Hyper-Parameter	Options
N_estimator	list(range(1,100))
Depth	list(range(10,20))

Table 2.3: RF Hyper-Parameter Option

Compare 3 Models

All 3 models were trained, with some of the models being tested twice, one ‘gridSearch’ and one ‘Trial & Error’. The evaluation method used to compare these models was borrowed from the week 3 practical, which allow for the ‘f1-scores’ and confusion matrix to be created for each model tested. [6] The comparison between all tested models are seen in Table 2.4 below, which shows the ‘F1 score’ and the corresponding confusion matrix.

	Hyper-Parameter	F1	Confusion Matrix
KNN ‘gridSearch’	'n_neighbors': 19 'weights': 'distance' 'metric': 'minkowski'	0.80	Figure 2.1
KNN ‘Trial & Error’	'n_neighbors': 4 'weights': 'distance' 'metric': 'minkowski'	0.85	Figure 2.2
SVM ‘gridSearch’	'C': 10 'kernel': 'poly' 'degree': 15	0.81	Appendix 2A
RF ‘gridSearch’	'n_estimators': 59 'max_depth': 10	0.81	Appendix 2B
RF ‘Trial & Error’	'n_estimators': 30 'max_depth': 5	0.83	Appendix 2C

Table 2.4: 3-Models Comparison

As seen in the table above, the models that used ‘Trial & Error’ method seemed to have resulted in higher ‘F1’ scores than the ‘gridSearch’ method. The highest ‘F1’ score was the KNN ‘Trial & Error’ model which had a score of 0.85. the next best was the RF ‘Trial & Error’ model which had a score of 0.83. The RF and SVM models using the ‘gridSearch’ method seem to have the same ‘F1’ scores, which seems to have occurred by chance. Finally, the least accurate model was the KNN ‘gridSearch’ model with an ‘F1’ score of 0.80. All these scores are still reasonable and there is little variance between the best and the worst models in terms of ‘F1’ scores. Going into detail about the variance between both the KNN models can be seen in Figure 2.1 and 2.2, which are the confusion matrices.

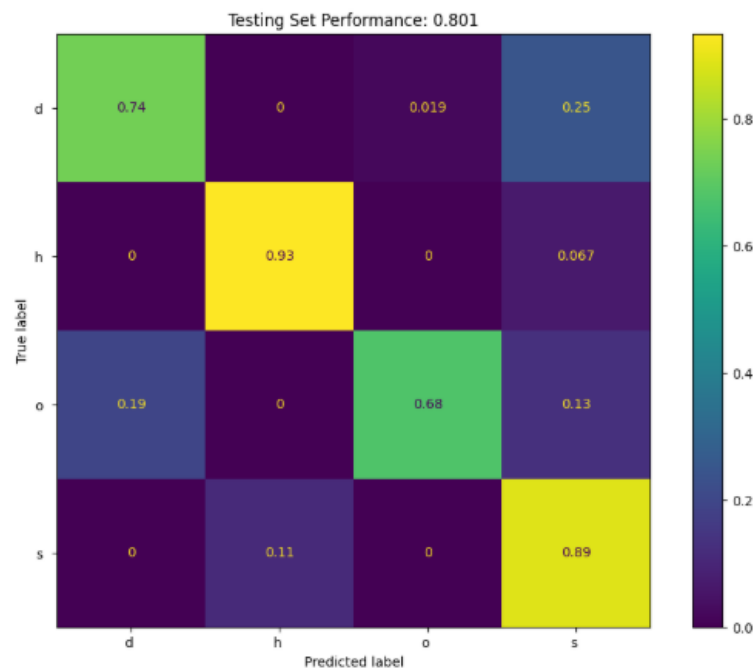


Figure 2.1: KNN (gridSearch) Confusion Matrix

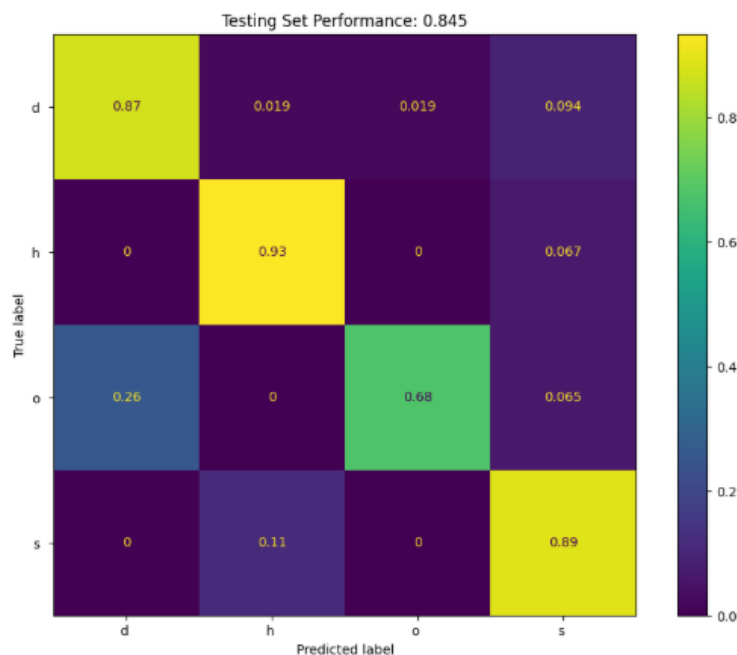


Figure 2.2: KNN (Trial & Error) Confusion Matrix

As seen in Figure 2.1 and 2.2 above, the confusion matrices between the two different methods are very similar. As seen in the figures, the ‘S-S’, ‘O-O’, ‘H-H’ grids are the same but change when it comes to the ‘D-D’ grid. The ‘Trial & Error’ method has ‘D-D’ accuracy of 0.87, whilst the ‘gridSearch’ method has a ‘D-D’ accuracy of 0.74. This happens because the ‘Trial & Error’ method corrects the mistake made in the ‘gridSearch’ model, which is the ‘D-S’ grid. In the ‘gridSearch’ model the ‘D-S’ grid is 0.25, whilst in the ‘Trial & Error’ model the grid is 0.094, a huge improvement on the mistake made by the ‘gridSearch’ model. Although this problem was discovered and fixed between models, both models still have one problem with them, which is the ‘O-D’ and ‘S-H’ grids. These grids still have a major number meaning there is still problems with both models. This problem can also be discovered in the difference between the two RF models (See Appendix 2B and 2C)

PROBLEM 3:

Pre-Processing & Neural Network Design:

The only processing was to minimise the number class in the training labels to 10 labels as the data needs to be classified as numbers ranging from 0 to 9, not 49.

The Deep Convolutional Neural Network (DCNN) was designed to be a simple VVG-style model. A VVG-style was chosen as it was an easy model to understand and modify. This network consist of the 3 convolutional blocks increasing from 128 to 256 to 512 with the 1st block learning patterns, the 2nd block leaning shapes and the 3rd block learning features. These were chosen as the data set given was small and these numbers allow for more efficiency and accuracy. Each convolution block is followed by a batch normalisation and max pooling. Max pooling is used to help with feature extraction and reduce overfitting. After these blocks the extracted image layers are then converted to 1D array via flattening, then are used within the dense layer which allow for the model to learn even more features about the images. The last two layers were the dropout and ‘softmax’ where the dropout layer would help with overfitting and the ‘softmax’ would help with multiple different classifications the model would be assessed on [9] [10].

Both DCNN models are compiled using the Adam optimiser for adaptive learning rates and categorical cross-entropy loss for allowing the models to display the results of each classes probabilities [11] [12]. Both models were trained with the same batch size, 32, and number of epochs, 100, with an ‘Early Stopping’ parameter that monitor the validation loss and has a patience value of 25. An ‘Early Stopping’ parameter allows the training to finish before 100 epochs if the validation loss starts to settle/converge, thus the accuracy of the model converges

Data Augmentations:

There were two types of data augmentations made to the data set. The first augmentation was the about rotating images 30 degrees left or right. This augmentation was done as going through the multiple other augmentation options it was discovered that particular numbers within particular fonts looked different based on the angles they were rotated, particularly the numbers 1 and 7. As seen below in Figure 3.3 some rotating the number 1 clockwise it could be considered a 7 and if a 7 was rotated anticlockwise it could be considered a 1. The other augmentation that was used was the inverted colour, as maybe when colour is inverted, the colours of particular photo may blend together, thus making it hard for the model.



Figure 3.1: Rotation Augmentation Example (Left = 7, Right = 1)

Compare 3 Models:

The SVM and the 2 DCNN models were trained and tested. The evaluation method used to assess the models was taken from the week 4 & 5 practical. This evaluation method allowed for the separate epochs and their loss and accuracy values to be printed out. This method also provided the Training vs Validation plots to be created for both accuracy and loss. The final information that this method also displays is the confusion matrix to allow for the analysis of the overall model. The comparison between the 3 models can be seen in Table 3.1 below, which shows the number of epochs, the 'F1' scores, training time, testing time and their corresponding confusion matrices.

Models	Epochs	F1	Trian Time	Test Time	Confusion Matrix
SVM	NONE	0.19	4.37	11.17	Train = Appendix 3A Testing = Figure 3.2
DCNN	68/100	0.83	52.54	6.53	Train = Appendix 3B Testing = Figure 3.3
Augmented DCNN	100/100	0.55	58.72	5.79	Train = Appendix 3C Testing = Figure 3.4

*Table 3.1: 3-Model Comparison
(Google Colab T4 GPU with available compute units)*

As seen in Table 3.1 above, the best model was the DCNN model with the unaugmented training data, with the next best being the DCNN with augmented data and the SVM model being the worst. Although the unaugmented DCNN had the highest training time, it got the most accurate result with a 0.83 and without overfitting the model entirely [13]. The unaugmented DCNN model could be modified to allow for a quicker training time but lose some accuracy. The augmented DCNN has an 'F1' score of 0.55, which is a reasonable score considering the augmented data that it was trained on. The SVM model was considered a failure as the model was chaotic in classifying the true positives.

As seen Figures 3.2, 3.3 and 3.4, the unaugmented DCNN has a near-perfect diagonal line from top left to bottom right, with some double digit numbers in the other corners. The augmented DCNN can only classify 1,2,4 and 5 properly and then has trouble classifying the rest of the numbers correctly. And finally the SVM confusion matrix shows that the model is a waste of time and not worth using when the multi-classification range is large.



Figure 3.2: SVM Testing Confusion Matrix

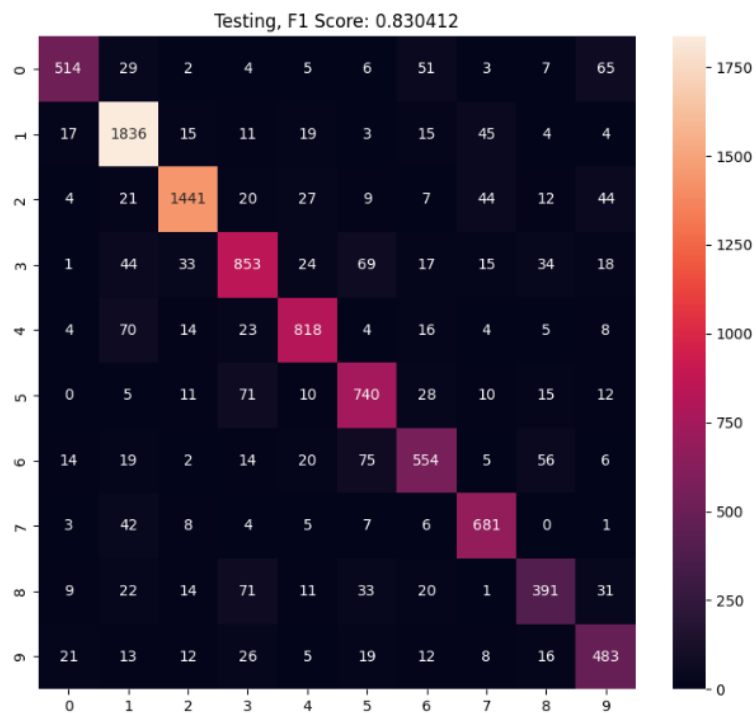


Figure 3.3: DCNN Testing Confusion Matrix

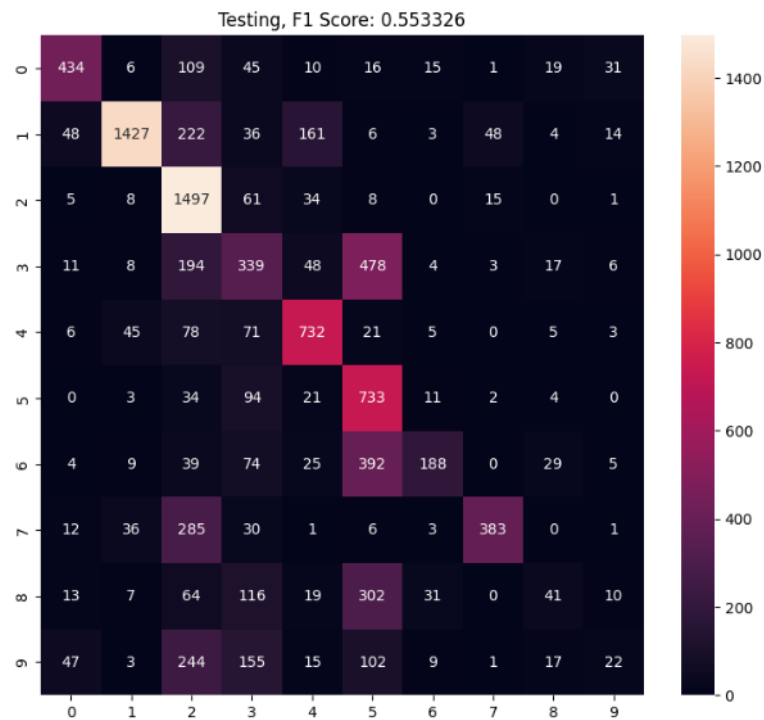


Figure 3.4: Augmented DCNN Testing Confusion Matrix

As seen in Figures 3.5 and 3.6, the unaugmented DCNN models' validation accuracy starts at 0.1 and rises about 0.2 every 10 epochs until about epochs 35, where it begins to settle to around 0.8 to 0.7 roughly. The augmented DCNN models' validation accuracy was quite the opposite where it flatlined around 0.2 accuracy for the first 40 epochs before learning and starting to increase to 0.6 around the 100 epochs mark. (See Appendix 3D & 3E for Train vs Validation Loss for DCNN models)

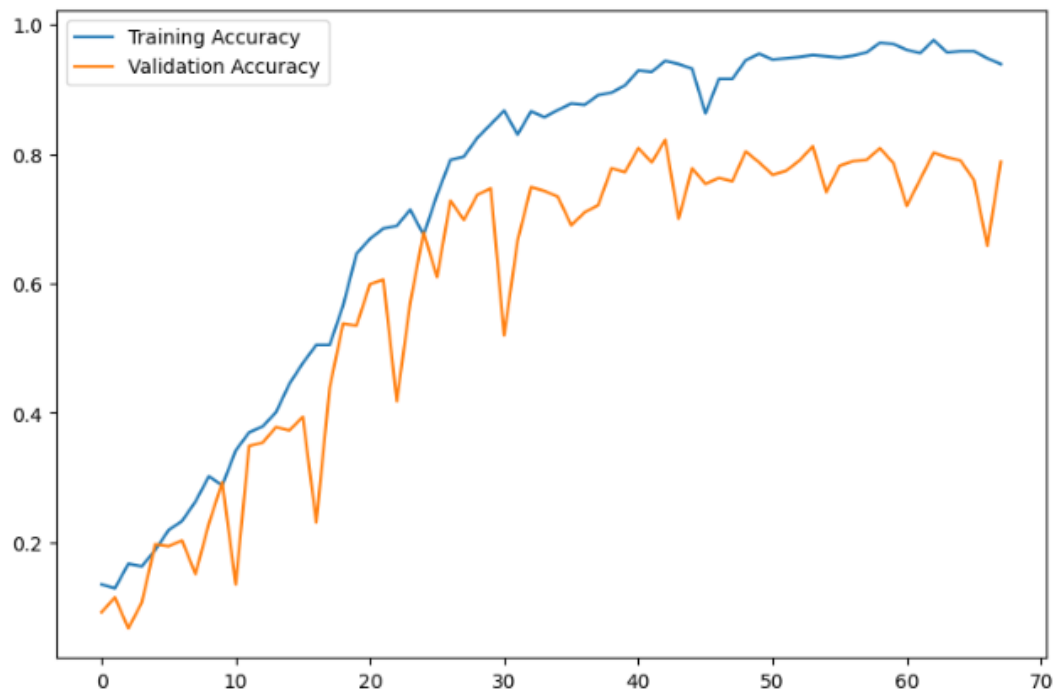


Figure 3.5: DCNN Training vs Validation Accuracy

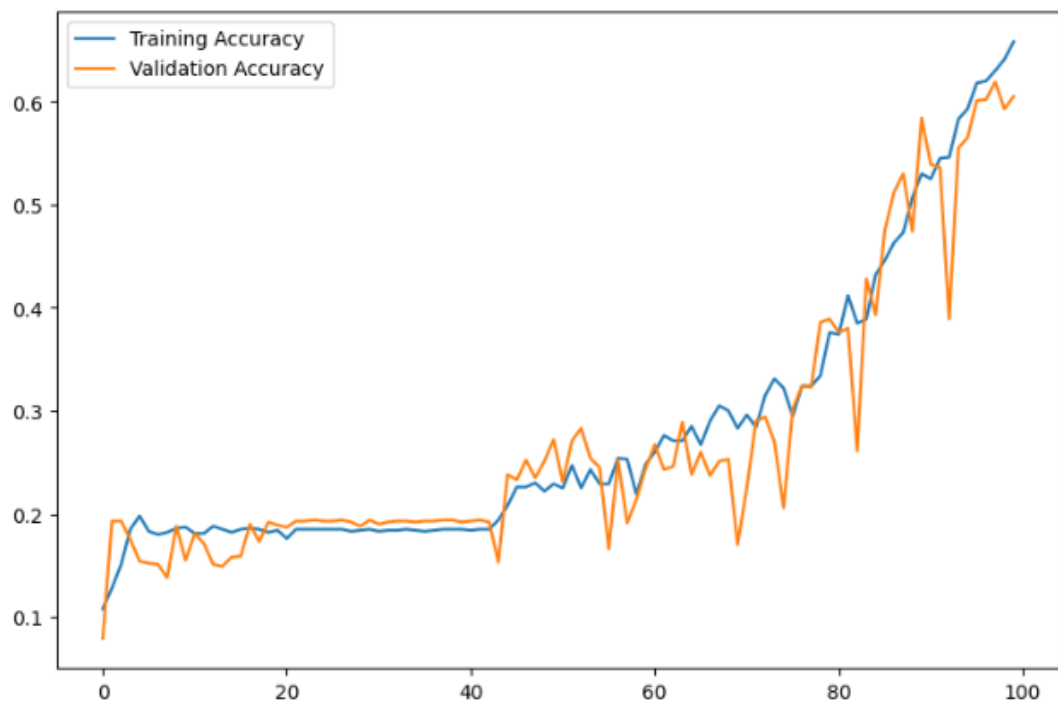


Figure 3.6: Augmented DCNN Training vs Validation Accuracy

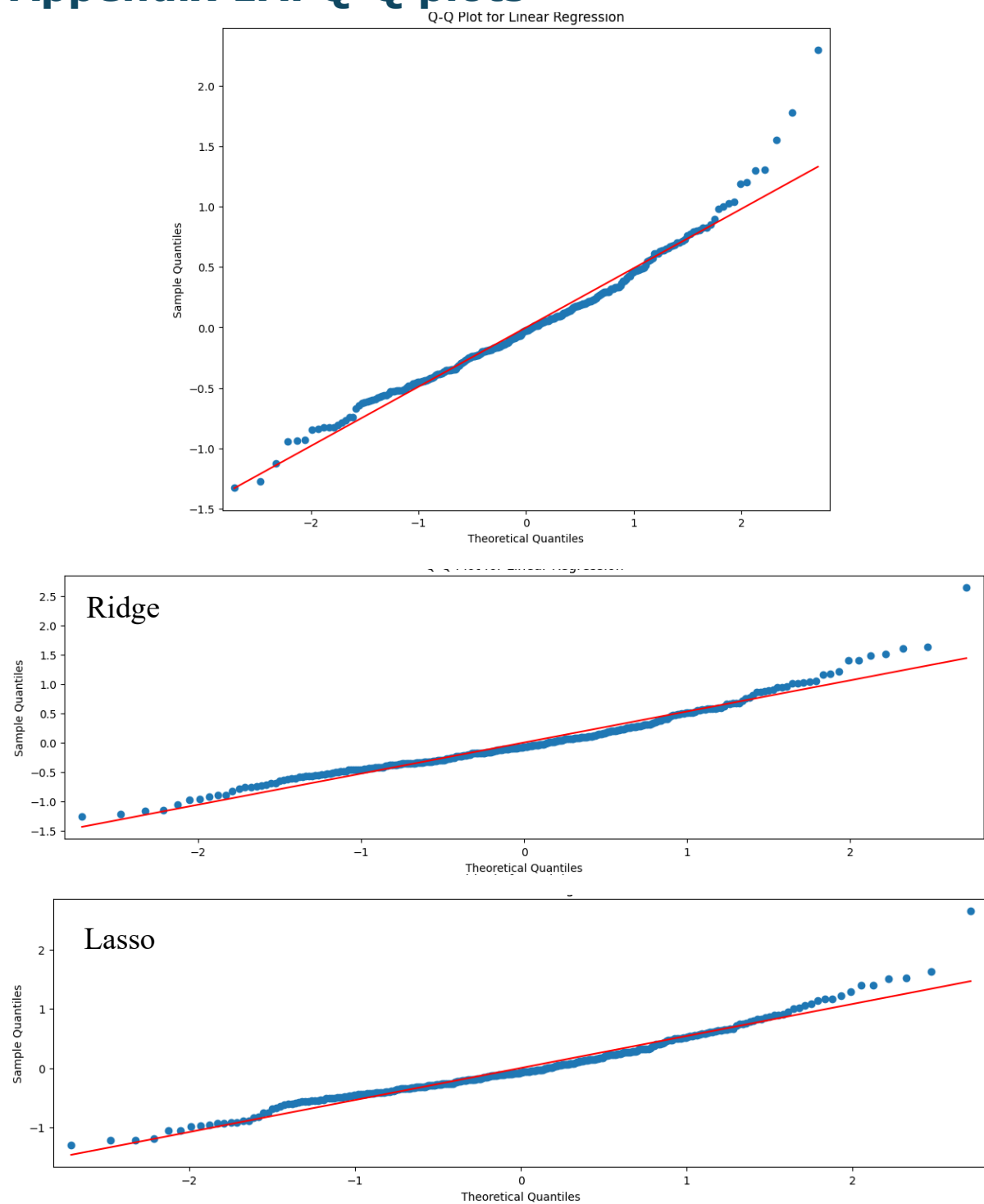
References:

- [1] S. Denman, “CAB420, Practical 2 - Practical Solution,” QUT, [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-2-practical-solutions?module_item_id=1798256. [Accessed 01 April 2025].
- [2] Z. Jaabi, “Data Standardization: How to Do It and Why It Matters,” builtin, 09 January 2025. [Online]. Available: <https://builtin.com/data-science/when-and-why-standardize-your-data#:~:text=Before%20Lasso%20and%20Ridge%20Regressions&text=Coefficients%20of%20variables%20with%20a,required%20before%20fitting%20both%20regressions..> [Accessed 01 April 2025].
- [3] S. Denman, “CAB420, Week 1 - Practical Solution,” QUT, [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-1-practical-solutions?module_item_id=1798249. [Accessed April 01 2025].
- [4] S. Olumide, “Root Mean Square Error (RMSE) In AI: What You Need To Know,” arize, 08 August 2023. [Online]. Available: <https://arize.com/blog-course/root-mean-square-error-rmse-what-you-need-to-know/#:~:text=Low%20RMSE%20values%20show%20that,mistakes%20and%20fewer%20accurate%20forecasts..> [Accessed 01 April 2025].
- [5] IGNEsa, “Crime Rates are Resilient,” IGNEsa, [Online]. Available: <https://ignesa.com/insights/crime-rate-and-ethical-crime-prediction/>. [Accessed 01 April 2025].
- [6] S. Denman, “CAB420, Practical 3 - Practical Solution,” QUT, [Online]. Available: https://canvas.qut.edu.au/courses/20364/pages/week-3-practical-solutions?module_item_id=1798263. [Accessed 02 April 2025].
- [7] Xbeat, “Mastering K-Nearest Neighbors Hyperparameters in Python,” Github, 12 September 2024. [Online]. Available: <https://github.com/xbeat/Machine-Learning/blob/main/Mastering%20K-Nearest%20Neighbors%20Hyperparameters%20in%20Python.md>. [Accessed 02 April 2025].

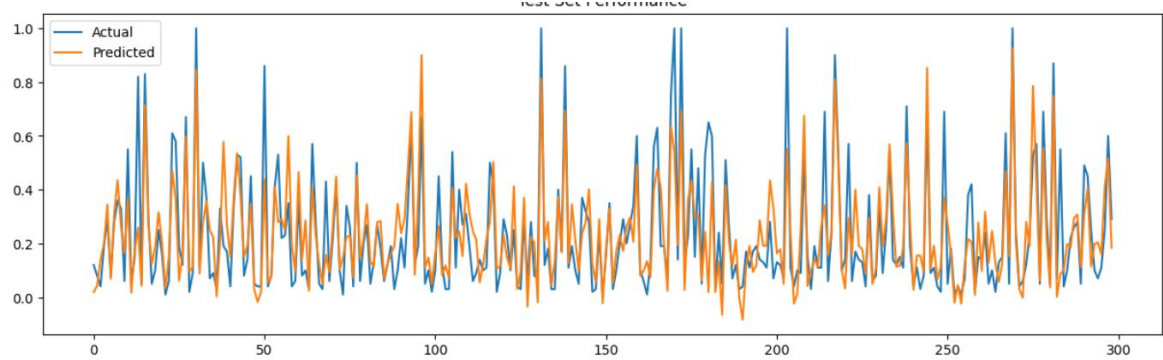
- [8] M. B. Fraj, “In Depth: Parameter tuning for Random Forest,” Medium, 22 December 2017. [Online]. Available: <https://medium.com/all-things-ai/in-depth-parameter-tuning-for-random-forest-d67bb7e920d>. [Accessed 02 April 2025].
- [9] P. Belagatti, “Understanding the Softmax Activation Function: A Comprehensive Guide,” SingleStore, 12 March 2024. [Online]. Available: <https://www.singlestore.com/blog/a-guide-to-softmax-activation-function/#:~:text=Softmax%20provides%20a%20well%2Ddefined,debugging%20of%20the%20neural%20network..> [Accessed 03 April 2025].
- [10] H. Yadav, “Dropout in Neural Networks,” towards data science, 05 July 2022. [Online]. Available: <https://towardsdatascience.com/dropout-in-neural-networks-47a162d621d9/>. [Accessed 05 April 2025].
- [11] R. Agarwal, “Complete Guide to the Adam Optimization Algorithm,” BuiltIn, 13 September 2023. [Online]. Available: <https://builtin.com/machine-learning/adam-optimization>. [Accessed 05 April 2025].
- [12] M. Sorokin, “Categorical Cross-Entropy: Unraveling its Potentials in Multi-Class Classification,” Medium, 17 January 2024. [Online]. Available: <https://medium.com/@vergotten/categorical-cross-entropy-unraveling-its-potentials-in-multi-class-classification-705129594a01#:~:text=Advantages%20of%20Categorical%20Cross%2DEntropy,model's%20confidence%20in%20its%20decisions..> [Accessed 05 April 2025].

APPENDICES:

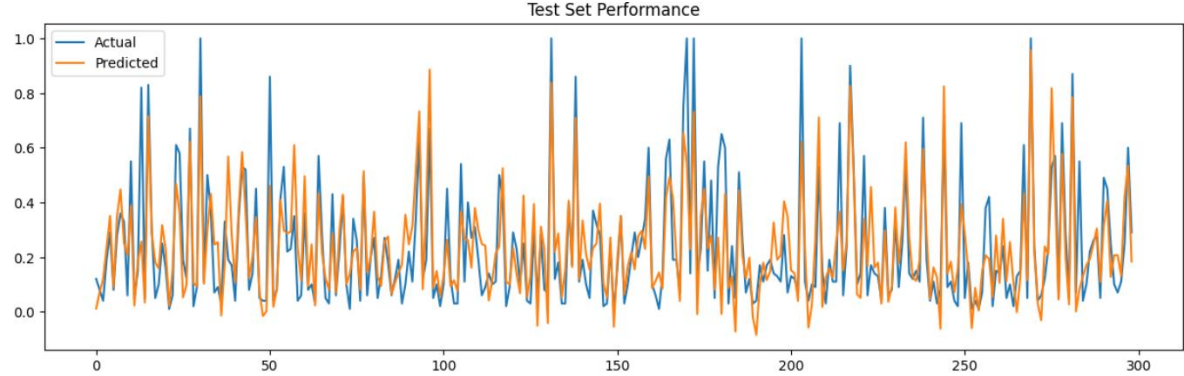
Appendix 1A: Q-Q plots



Appendix 1B: Ridge Test Set Performance



Appendix 1C: Lasso Test Set Performance



Appendix 1D: Linear Regression Summary

Model LR RMSE = 0.7297

Model LR R² Score = 0.7591

OLS Regression Results			
=====			
Dep. Variable:	ViolentCrimesPerPop	R-squared (uncentered):	0.759
Model:	OLS	Adj. R-squared (uncentered):	0.637
Method:	Least Squares	F-statistic:	6.239
Date:	Sat, 05 Apr 2025	Prob (F-statistic):	4.63e-28
Time:	09:35:57	Log-Likelihood:	-210.77
No. Observations:	298	AIC:	621.5
Df Residuals:	198	BIC:	991.2
Df Model:	100		
Covariance Type:	nonrobust		
=====			

Appendix 1E: Ridge Summary

RIDGE:
Best values on Validation Data set
Best Alpha =0.06135907273413163
Best R Squared = 0.7182056062081854
Best Adjusted = 0.5751627667199547
Best RMSE (val) = 0.14247714909139023
Best RMSE (test) = 0.1314902865929718
Best coefficients on the normalised model

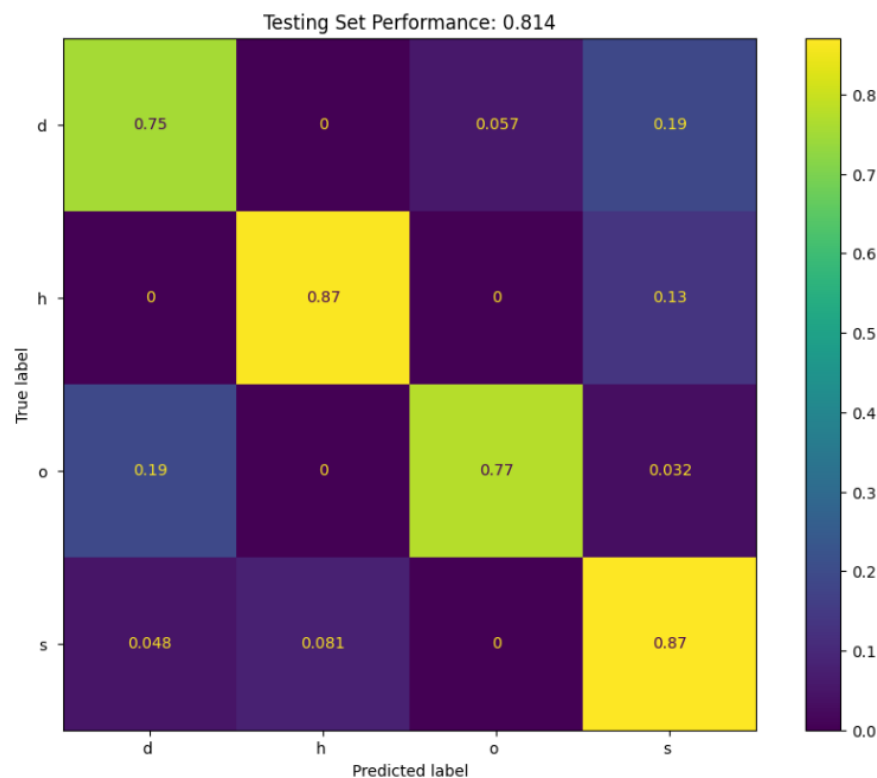
Appendix 1F: Lasso Summary

LASSO:
Best values on Validation Data set
Best Alpha =0.002364489412645407
Best R Squared = 0.7062344312663479
Best Adjusted = 0.5571148532289609
Best RMSE (val) = 0.14390977923618642
Best RMSE (test) = 0.1319091428887659
Best coefficients on the normalised model

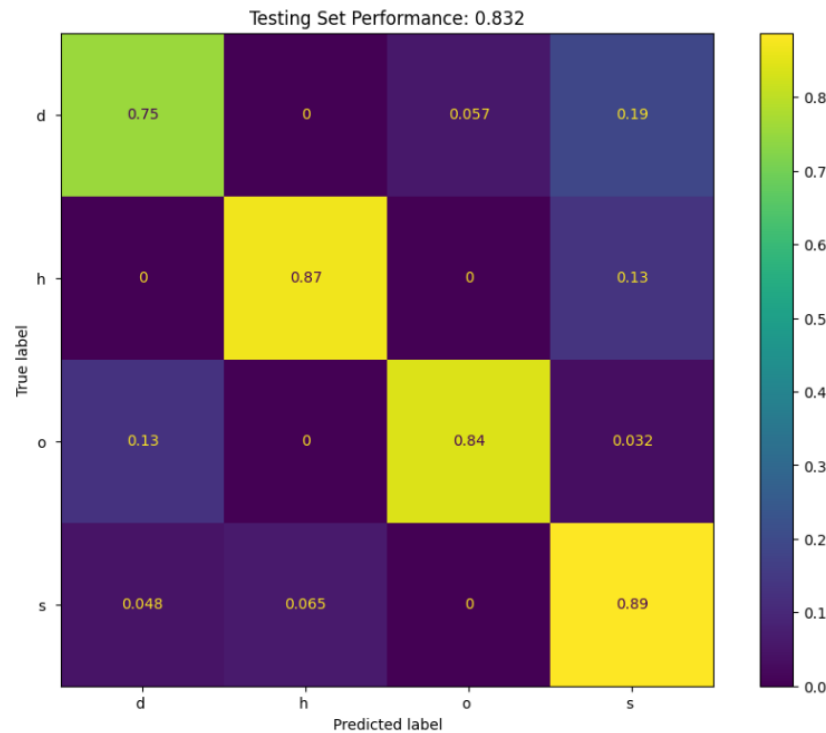
Appendix 2A: SVM Grid Confusion Matrix



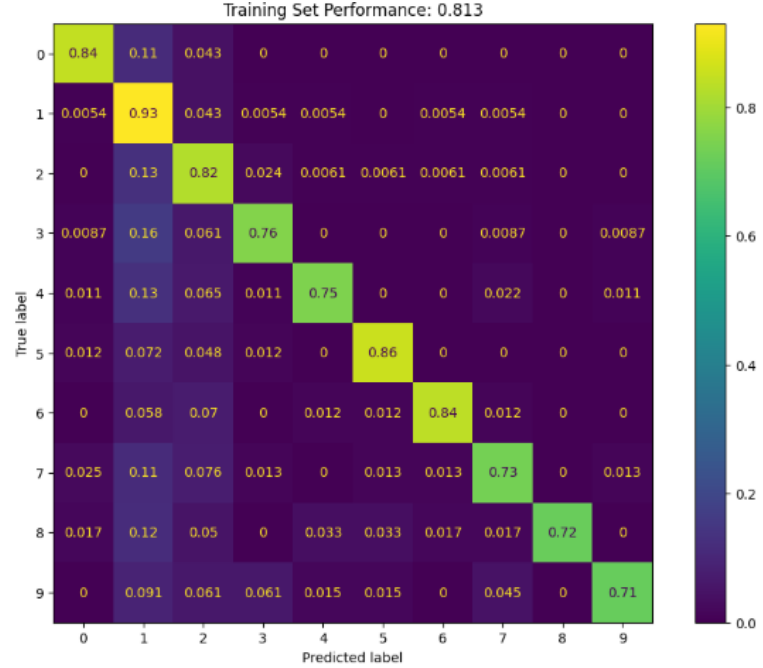
Appendix 2B: RF Grid Confusion Matrix



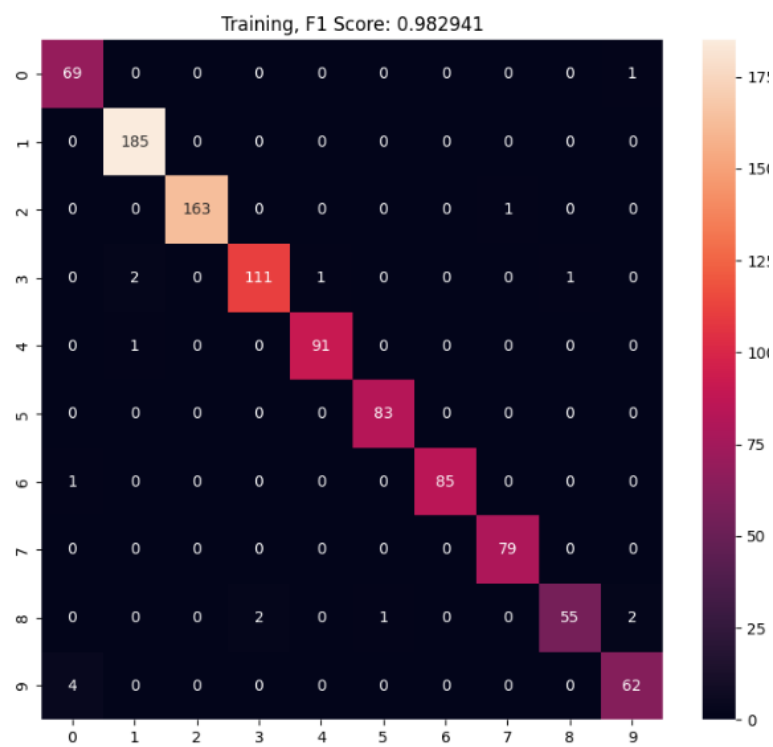
Appendix 2C: RF Trial Confusion Matrix



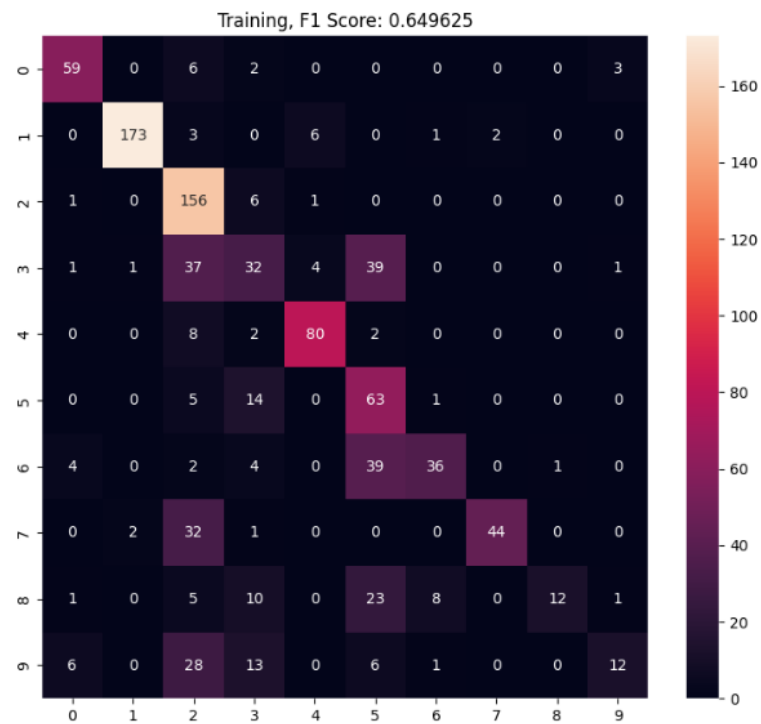
Appendix 3A: SVM training Confusion Matrix



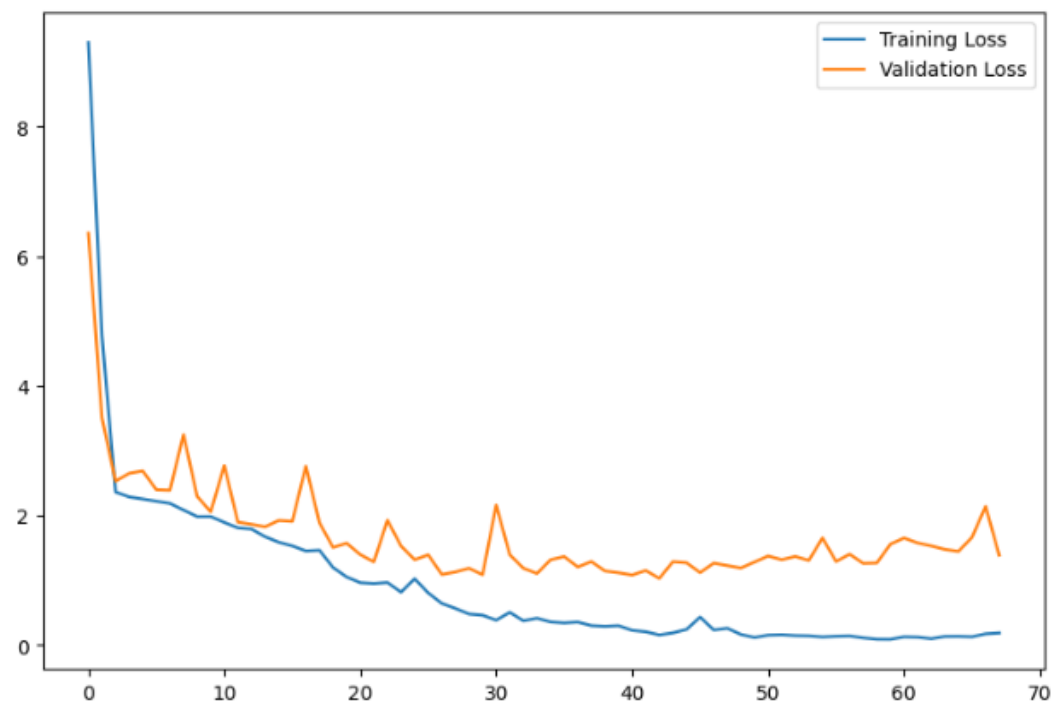
Appendix 3B: DCNN training Confusion Matrix



Appendix 3C: Augmented DCNN training Confusion Matrix



Appendix 3D: DCNN Training vs Validation Loss



Appendix 3E: Augmented DCNN Training vs Validation Loss

